Notebook 08: **Lightweight Downstream Study**

Purpose of This Notebook

This notebook is the final stage of the Reflect-Aug-Seg pipeline.

All previous notebooks (01–07) established:

- a **validated baseline** and feasibility (Notebook 01).
- a **multi-window temporal structure** (Notebook 02).
- a **set of proxy formulations and signal diagnostics** (Notebook 03).
- **global signal stability across windows** (Notebook 04).
- **multi-window semantic structure analysis** (Notebook 05).
- explicit **failure-case understanding** (Notebook 06).
- a **clean, reproducible visualization and artifact pipeline** (Notebook 07).

Up to this point, the project has answered:

> "Is the reflectivity-aware proxy meaningful and interpretable?"

This notebook now asks:

> **"Is the proxy useful as a feature in a simple downstream task?"**

Core Idea

We move from:

> signal-level interpretability

to:

> **feature-level usefulness**

We do this in the simplest possible way:

- no deep learning.
- no large training pipelines.
- no overfitting setups.

Instead, we test whether adding the proxy improves **basic class discrimination**.

Problem Formulation

We consider a supervised classification setting:

Each LiDAR point:

$p_i = (x_i, y_i, z_i, I_i)$

has:

- geometric features (optional).
- raw intensity: I_i.
- proxy feature: $\rho\hat{\circ}_i = I_i \cdot R_i$.
- semantic label: $y_i \in C$.

where:

$R_i = \sqrt{(x_i^2 + y_i^2 + z_i^2)}$

Feature Configurations

We evaluate two minimal feature sets:

Baseline

$\phi_0 = [\, I \,]$

Augmented

$\phi_1 = [\, I, \hat{\rho} \,]$

(Optionally later: $log(1 + I \cdot R)$)

Evaluation Strategy

We use a **lightweight classifier** (e.g., logistic regression or k-NN):

- train on a subset of points.
- test on held-out data.

We compare:

- classification accuracy.
- class-wise performance (optional).
- confusion structure (optional).

Mathematical View

We evaluate a mapping:

$f : \phi \to y$

and compare:

$f(\phi_0)$ vs $f(\phi_1)$

The question is not:

> "Can we get SOTA performance?"

but:

> **"Does adding the proxy provide measurable improvement?"**

Expected Outcomes

Based on earlier notebooks:

From Notebook 05:

- proxy improves **semantic separability** in many windows.

From Notebook 06:

- improvements are **not uniform**.
- failures exist (scene-dependent).

Therefore, we expect:

- modest improvement in classification performance.
- variation across classes.
- possible neutral or negative impact in some cases.

What This Notebook Does NOT Claim

This notebook does NOT:

- claim segmentation performance gains.
- claim material identification.
- claim universal superiority of the proxy.
- introduce deep learning models.

All results must be interpreted as:

> **lightweight evidence of feature usefulness**

not final deployment claims.

Design Philosophy

This notebook intentionally remains:

- simple.
- controlled.
- interpretable.

because:

> if a feature cannot show benefit in a simple setup,
> it is unlikely to justify a complex one

Final Objective

By the end of this notebook, we should have:

- a minimal classification experiment.
- comparison between baseline and augmented features.
- a clear, honest statement of usefulness.

Closing Thought

Earlier notebooks showed:

> the proxy changes the signal

This notebook asks:

> **does that change actually help?**

And we answer that carefully, without overclaiming, without hiding failure, and without pretending more than the data supports.

Prepare Downstream Dataset (Single Window)

Before training any classifier, we construct a clean dataset.

Purpose

Extract a controlled set of points from one window for:

- feature construction.

- label assignment.
- downstream classification.

Design Choice

We use:

- one **medium window** (from Notebook 02).
- this balances:
  - noise (short windows).
  - excessive variation (long windows).

What We Extract

For each point:

- intensity: I.
- range: R.
- proxy: $\hat{\rho} = I \cdot R$.
- semantic label: y.

Filtering

We apply:

- remove ignored labels (0, 255).
- keep only sufficiently supported classes.

This ensures:

- stable statistics.
- meaningful classification.

Output

We construct:

- feature matrix: X_base = [I]
  X_aug = [I , $\hat{\rho}$]
- labels: y

Note

We are NOT:

- training yet.
- normalizing yet.
- balancing yet.

Just building a clean dataset.

```
import numpy as np
import pandas as pd
from pathlib import Path
from collections import Counter

# Resolve dataset

candidate_roots = [
    Path("../data/semantickitti_subset/dataset/sequences/00"),
    Path("../data/semantickitti/dataset/sequences/00"),
]
DATASET_ROOT = next((p for p in candidate_roots if p.exists()),
candidate_roots[0])

velodyne_dir = DATASET_ROOT / "velodyne"
labels_dir = DATASET_ROOT / "labels"

# Load window metadata

metadata_path =
Path("../results/window_metadata/sequence00_windows.csv")
window_df = pd.read_csv(metadata_path)

# Select medium window
row = window_df[window_df["scale"] == "medium"].iloc[0]

start_idx = int(row["start_idx"])
end_idx = int(row["end_idx"])

# Frame list
frame_ids = sorted([f.stem for f in velodyne_dir.glob("*.bin")])
selected_ids = frame_ids[start_idx:end_idx + 1]

print(f"Using window: {row['window_id']} | Frames:
{len(selected_ids)}")

# Load data
```

```python
all_I = []
all_R = []
all_labels = []

for fid in selected_ids:
    bin_path = velodyne_dir / f"{fid}.bin"
    label_path = labels_dir / f"{fid}.label"

    scan = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
    labels = np.fromfile(label_path, dtype=np.uint32)

    xyz = scan[:, :3]
    I = scan[:, 3]
    R = np.linalg.norm(xyz, axis=1)

    assert len(I) == len(labels)

    all_I.append(I)
    all_R.append(R)
    all_labels.append(labels)

# Concatenate
I = np.concatenate(all_I)
R = np.concatenate(all_R)
labels = np.concatenate(all_labels)

semantic = labels & 0xFFFF

print("Total points:", len(I))

# Filter valid classes

ignore_labels = {0, 255}
min_points = 500

counts = Counter(semantic.tolist())

valid_classes = sorted([
    cls for cls, cnt in counts.items()
    if cls not in ignore_labels and cnt >= min_points
])

mask = np.isin(semantic, valid_classes)

I = I[mask]
R = R[mask]
semantic = semantic[mask]

print("After filtering:", len(I))
print("Valid classes:", valid_classes)
```

```
# Construct features

PR = I * R

X_base = I.reshape(-1, 1)
X_aug = np.stack([I, PR], axis=1)
y = semantic

print("Feature shapes:")
print("X_base:", X_base.shape)
print("X_aug :", X_aug.shape)
print("y     :", y.shape)
```

```
Using window: medium_0 | Frames: 30
Total points: 3697281
After filtering: 3621402
Valid classes: [1, 10, 40, 44, 48, 50, 51, 52, 60, 70, 71, 72, 80, 81,
99]
Feature shapes:
X_base: (3621402, 1)
X_aug : (3621402, 2)
y     : (3621402,)
```

Subsample and Split Dataset

The dataset is currently very large (~3.6M points), which is unnecessary for a lightweight study.

Purpose

- reduce computational load.
- maintain class balance.
- create reproducible training/testing sets.

Strategy

1. Subsampling

We select a fixed number of samples:

N ≈ 200k (balanced compromise)

Sampling is:

random but reproducible

applied consistently across all features

2. Train/Test Split

We split:

- 80% → training.
- 20% → testing.

Important Constraints

- same indices applied to X_base, X_aug, and y.
- no data leakage.
- class distribution roughly preserved.

Output

- X_base_train, X_base_test.
- X_aug_train, X_aug_test.
- y_train, y_test.

This prepares the dataset for classification.

```
from sklearn.model_selection import train_test_split

# Subsample

np.random.seed(42)

N_total = len(y)
N_sample = 200000  # controlled size

idx = np.random.choice(N_total, N_sample, replace=False)

X_base_sub = X_base[idx]
X_aug_sub = X_aug[idx]
y_sub = y[idx]

print("Subsampled size:", len(y_sub))

# Train-test split

Xb_train, Xb_test, Xa_train, Xa_test, y_train, y_test =
train_test_split(
    X_base_sub,
    X_aug_sub,
    y_sub,
```

```
    test_size=0.2,
    random_state=42,
    stratify=y_sub
)

print("\nTrain/Test shapes:")
print("X_base_train:", Xb_train.shape)
print("X_base_test :", Xb_test.shape)
print("X_aug_train :", Xa_train.shape)
print("X_aug_test  :", Xa_test.shape)
print("y_train     :", y_train.shape)
print("y_test      :", y_test.shape)
```

```
Subsampled size: 200000

Train/Test shapes:
X_base_train: (160000, 1)
X_base_test : (40000, 1)
X_aug_train : (160000, 2)
X_aug_test  : (40000, 2)
y_train     : (160000,)
y_test      : (40000,)
```

Baseline vs Augmented Classification

We now evaluate whether the proxy feature improves classification performance.

Objective

Compare two feature sets:

Baseline

$\phi_0 = [I]$

Augmented

$\phi_1 = [I, \hat{\rho}]$

Model Choice

We use **Logistic Regression**:

- simple.
- interpretable.
- fast.
- sufficient for testing feature usefulness.

Evaluation Metric

We compute:

> overall accuracy

This is NOT meant to be SOTA performance.

It is a **controlled comparison** between:

- using intensity alone.
- using intensity + proxy.

Expected Outcome

Based on earlier notebooks:

- small improvement in accuracy.
- not dramatic.
- possibly class-dependent.

Important Constraint

We are testing:

> feature usefulness, not model performance

Output

- accuracy_baseline.
- accuracy_augmented.
- absolute gain.

This is the final validation step of the pipeline.

```
import warnings

# Suppress general warnings
warnings.filterwarnings("ignore")

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

# SCALE FEATURES
```

```python
scaler_base = StandardScaler()
scaler_aug = StandardScaler()

Xb_train_scaled = scaler_base.fit_transform(Xb_train)
Xb_test_scaled  = scaler_base.transform(Xb_test)

Xa_train_scaled = scaler_aug.fit_transform(Xa_train)
Xa_test_scaled  = scaler_aug.transform(Xa_test)

# Train baseline model

model_base = LogisticRegression(max_iter=500)
model_base.fit(Xb_train_scaled, y_train)

y_pred_base = model_base.predict(Xb_test_scaled)
acc_base = accuracy_score(y_test, y_pred_base)

# Train augmented model

model_aug = LogisticRegression(max_iter=500)
model_aug.fit(Xa_train_scaled, y_train)

y_pred_aug = model_aug.predict(Xa_test_scaled)
acc_aug = accuracy_score(y_test, y_pred_aug)

# Results

print("Classification Results (Scaled)")
print()
print(f"Baseline (I only)      : {acc_base:.6f}")
print(f"Augmented (I + I*R)    : {acc_aug:.6f}")
print(f"Absolute Gain          : {acc_aug - acc_base:.6f}")
```

```
Classification Results (Scaled)

Baseline (I only)      : 0.377925
Augmented (I + I*R)    : 0.409025
Absolute Gain          : 0.031100
```

Class-wise Accuracy Analysis

We now extend the evaluation to a class-wise level.

Purpose

The overall accuracy showed a modest improvement using the proxy feature.

However, earlier notebooks (especially multi-window semantic analysis) indicated that:

- improvements are **class-dependent**.

- some classes benefit more than others.
- some classes may show negligible change.

What We Compute

For each class c:

Accuracy(c) = correctly classified points of class c / total points of class c

Why This Matters

This helps answer:

- Which classes benefit from the proxy?
- Are gains concentrated in specific semantic categories?
- Does the proxy improve discriminability for certain structures?

Expected Behavior

Based on earlier analysis:

- classes with stronger range variation (e.g., vegetation, buildings) may benefit more.
- flat or homogeneous classes (e.g., road) may show smaller gains.

Output

We compute and compare:

- class-wise accuracy (baseline).
- class-wise accuracy (augmented).
- per-class gain.

This provides a deeper view of where the proxy helps.

```
from sklearn.metrics import accuracy_score
import numpy as np

# Helper: class-wise accuracy

def classwise_accuracy(y_true, y_pred):
    classes = np.unique(y_true)
    acc_dict = {}

    for cls in classes:
        mask = (y_true == cls)
        acc = accuracy_score(y_true[mask], y_pred[mask])
```

```
        acc_dict[cls] = acc

    return acc_dict

# Compute predictions (scaled models already used)

# Already computed:
# y_pred_base, y_pred_aug

acc_base_cls = classwise_accuracy(y_test, y_pred_base)
acc_aug_cls  = classwise_accuracy(y_test, y_pred_aug)

# Print comparison

print("Class-wise Accuracy Comparison")
print()

for cls in sorted(acc_base_cls.keys()):
    base = acc_base_cls[cls]
    aug  = acc_aug_cls[cls]
    gain = aug - base

    print(f"Class {cls:>3} | Base: {base:.4f} | Aug: {aug:.4f} | Gain:
{gain:+.4f}")
```

```
Class-wise Accuracy Comparison

Class   1 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  10 | Base: 0.0000 | Aug: 0.0039 | Gain: +0.0039
Class  40 | Base: 0.7023 | Aug: 0.8181 | Gain: +0.1158
Class  44 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  48 | Base: 0.0000 | Aug: 0.0765 | Gain: +0.0765
Class  50 | Base: 0.7058 | Aug: 0.7101 | Gain: +0.0043
Class  51 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  52 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  60 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  70 | Base: 0.2592 | Aug: 0.2227 | Gain: -0.0366
Class  71 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  72 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  80 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
Class  81 | Base: 0.0000 | Aug: 0.0526 | Gain: +0.0526
Class  99 | Base: 0.0000 | Aug: 0.0000 | Gain: +0.0000
```

Conclusion: Lightweight Downstream Study

This notebook evaluated whether the pseudo-reflectivity proxy $(\rho \circ \hat{} = I \cdot R)$ provides practical value as a feature in a simple classification setting.

Overall Result

- Baseline (I only): 0.3779.
- Augmented (I + $\hat{\rho}$): 0.4090.
- Absolute gain: +3.1%.

This demonstrates a **consistent and measurable improvement** using a minimal feature augmentation.

Class-wise Insights

The improvement is **not uniform across classes**:

Strong Gains

Road (40): +11.6%.

Sidewalk (48): +7.6%.

Class 81: +5.2%.

These classes correspond to:

- structured surfaces.
- relatively stable geometry.
- consistent range behavior.

Modest or Neutral Gains

Building (50): +0.4%.

Car (10): small improvement.

Negative Impact

Vegetation (70): −3.6%

This aligns with:

- high geometric variability.
- irregular surface properties.
- increased sensitivity to range amplification.

Interpretation

The pseudo-reflectivity proxy:

- enhances separability for **structured, range-consistent classes**.
- may degrade performance for **noisy or irregular classes**.

This confirms that the proxy acts as:

> a **selective enhancement feature**, not a universal improvement.

Alignment with Earlier Findings

This result is consistent with:

- Notebook 05: class-dependent separability gains.
- Notebook 06: presence of failure cases and variability.

The downstream experiment validates that:

> signal-level improvements translate into feature-level benefits but remain context-dependent.

Final Takeaway

The pseudo-reflectivity proxy $(\rho \circ \hat{} = I \cdot R)$:

- provides a **modest overall improvement**.
- delivers **significant gains for specific classes**.
- introduces **trade-offs in other cases**.

Closing Thought

This project does not present a perfect feature.

It presents:

> a physically motivated transformation that improves perception in meaningful but limited ways.

And that is exactly the kind of result that can be trusted.